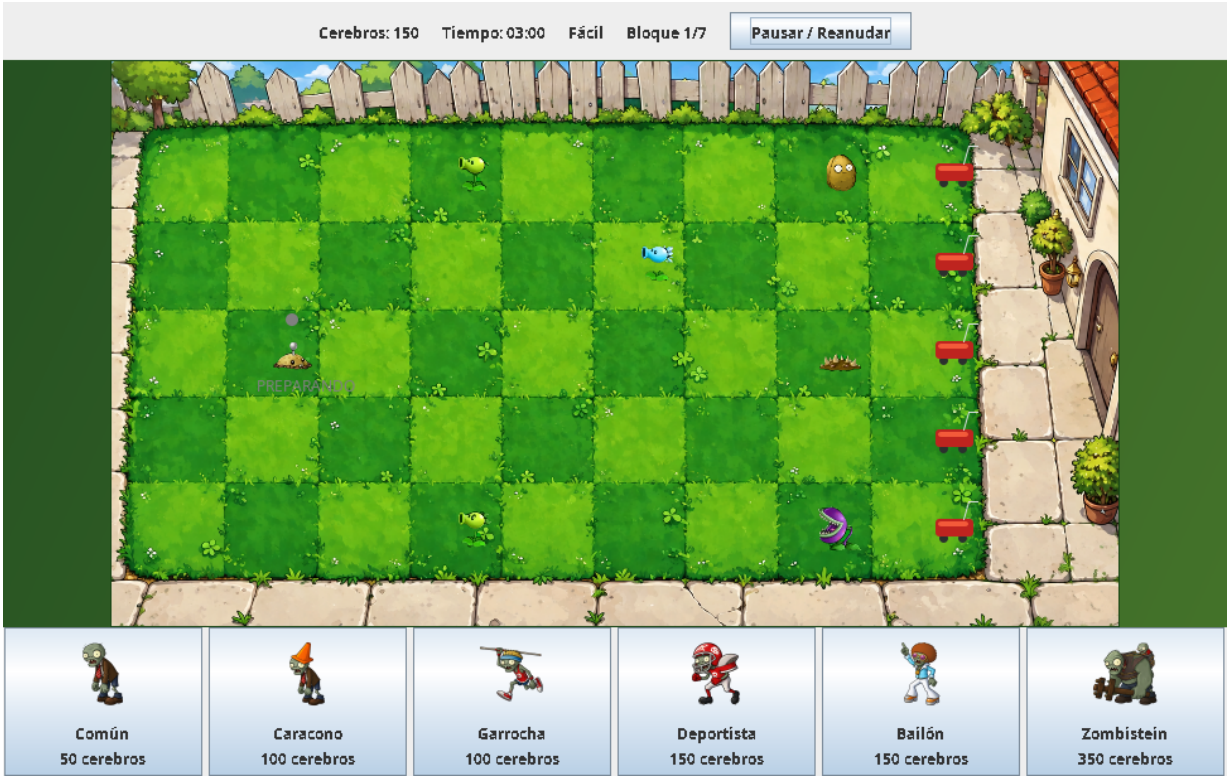
Zombis vs. Plantas

Manual técnico

Arquitectura, algoritmos, concurrencia, persistencia y validación de la versión corregida.

Introducción a la Programación y Computación 1
Centro Universitario de Occidente · USAC
Segundo semestre 2026



Versión de entrega con imágenes originales, controles compactos, estados gráficos y generadores con hilos independientes.

Repositorio: jesergarcia05-rgb/Proyecto-IPC-2-Plantas-Vs-Zombies
Rama: entrega-corregida-con-imagenes

1. Alcance y estructura del proyecto

Aplicación de escritorio Java Swing. El jugador compra zombis que avanzan de izquierda a derecha hacia una casa defendida por plantas. El jardín proviene de un JSON y cada cuenta mantiene partidas y estadísticas propias.

El proyecto compila con **JDK 21 estándar**. No usa bibliotecas Java externas, Collections dinámicas, Lombok, Timer de Swing, volatile, synchronized, wait/notify ni java.util.concurrent. Se usan arreglos nativos que duplican su capacidad cuando es necesario.

La documentación se genera fuera del juego con Python/ReportLab y Graphviz. Esas herramientas no son dependencias de compilación ni ejecución de la aplicación Java.

```
src/main/java/com/mycompany/proyecto2_plantasvszombies/
  principal/   Arranque de la aplicación
  vista/       Ventanas, paneles, recursos e hilos
  modelo/      Partida, jardín, combate, cuentas y datos
  plantas/     Jerarquía de defensas
  zombis/      Jerarquía de atacantes
  archivos/    JSON, cuentas, estadísticas y binarios
src/main/resources/Imagenes/
Niveles/     Archivos JSON de ejemplo
dist/        JAR ejecutable
tests/       Pruebas sin bibliotecas externas
herramientas/ Capturas y generación de manuales
```

Arranque y separación de responsabilidades

Proyecto2_PlantasVsZombies.main solo publica el arranque en SwingUtilities.invokeLater. Juego crea el gestor y abre VentanaInicio. Las ventanas interpretan acciones del usuario; Partida almacena estado; MotorCombate resuelve las reglas sin depender de Swing. Los gestores leen y escriben archivos.

2. UML del estado y del motor

El diagrama muestra datos persistentes y asociaciones. El rombo negro representa composición; una flecha simple, una referencia o uso. Los atributos llevan “−” y los métodos públicos “+”. El esquema omite getters repetidos para facilitar la lectura.

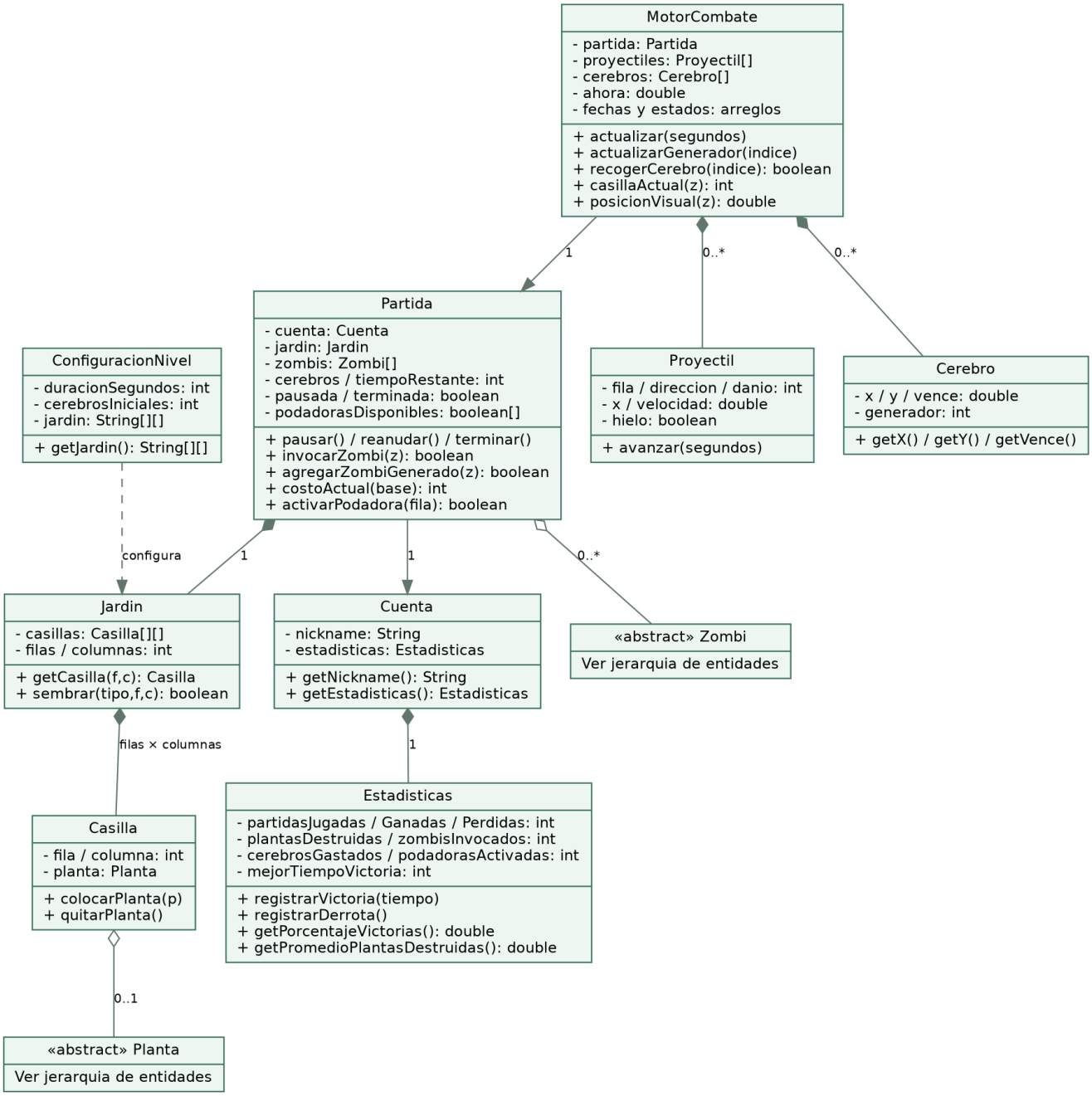


Figura T1. Modelo de cuentas, partida, jardín, proyectiles y recursos. Las fuentes DOT y PNG se incluyen en documentacion/diagramas.

3. UML de las entidades

Planta y Zombi son clases abstractas. Las clases concretas fijan salud, daño, costo y cadencias. El motor usa el tipo concreto cuando una mecánica exige un tratamiento espacial especial. Las flechas con triángulo vacío representan herencia.

Ejemplos de polimorfismo: recibirDanio descuenta primero un escudo en Caracono/Deportista; Pinchohierba ignora las mordidas; getDanio del Caracono retorna 15 después de perder el cono. Los atributos permanecen privados y los cambios de posición/velocidad auxiliares están protegidos en Zombi.

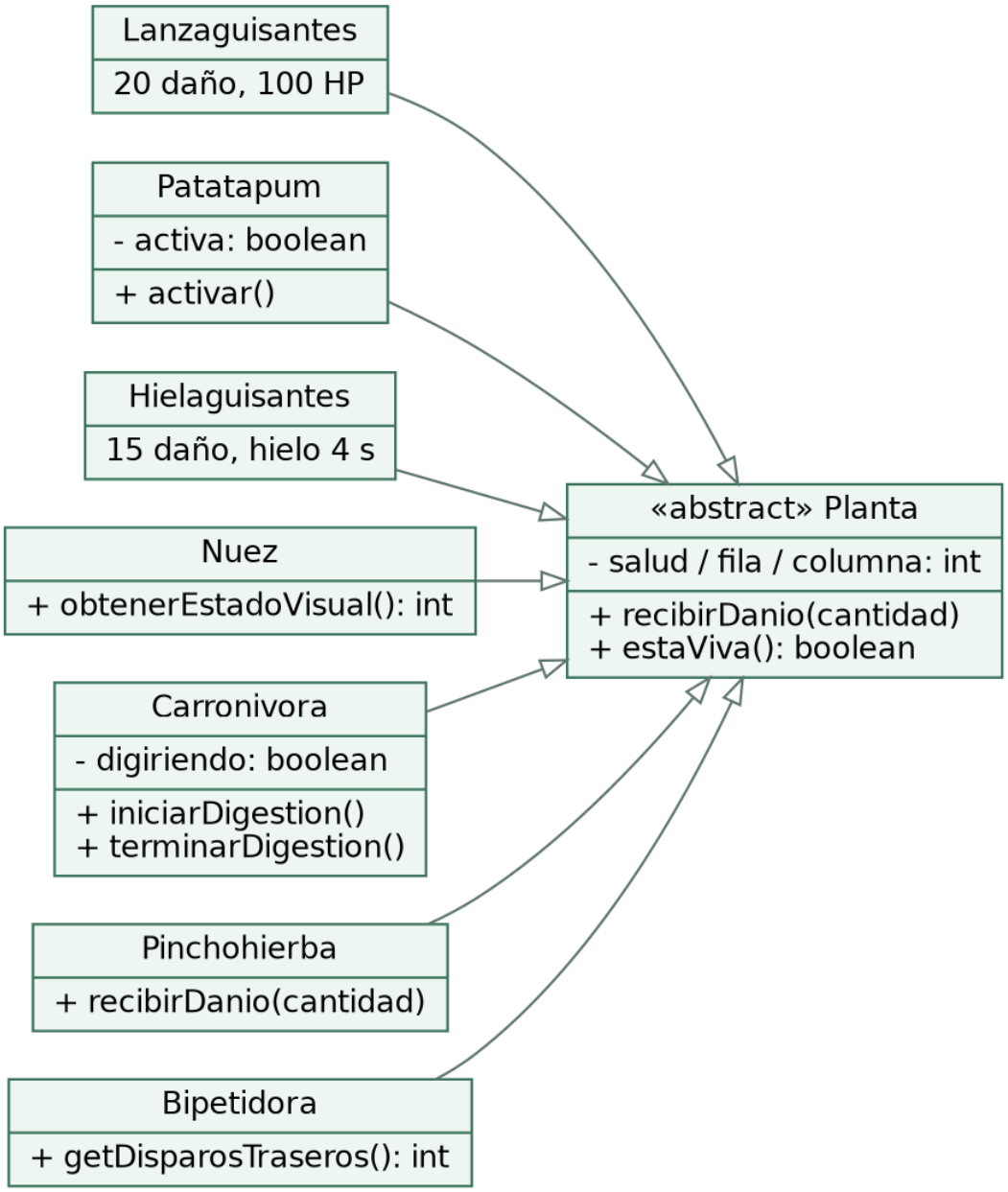


Figura T2a. Herencia de las siete plantas.

3.1. Jerarquía de zombis

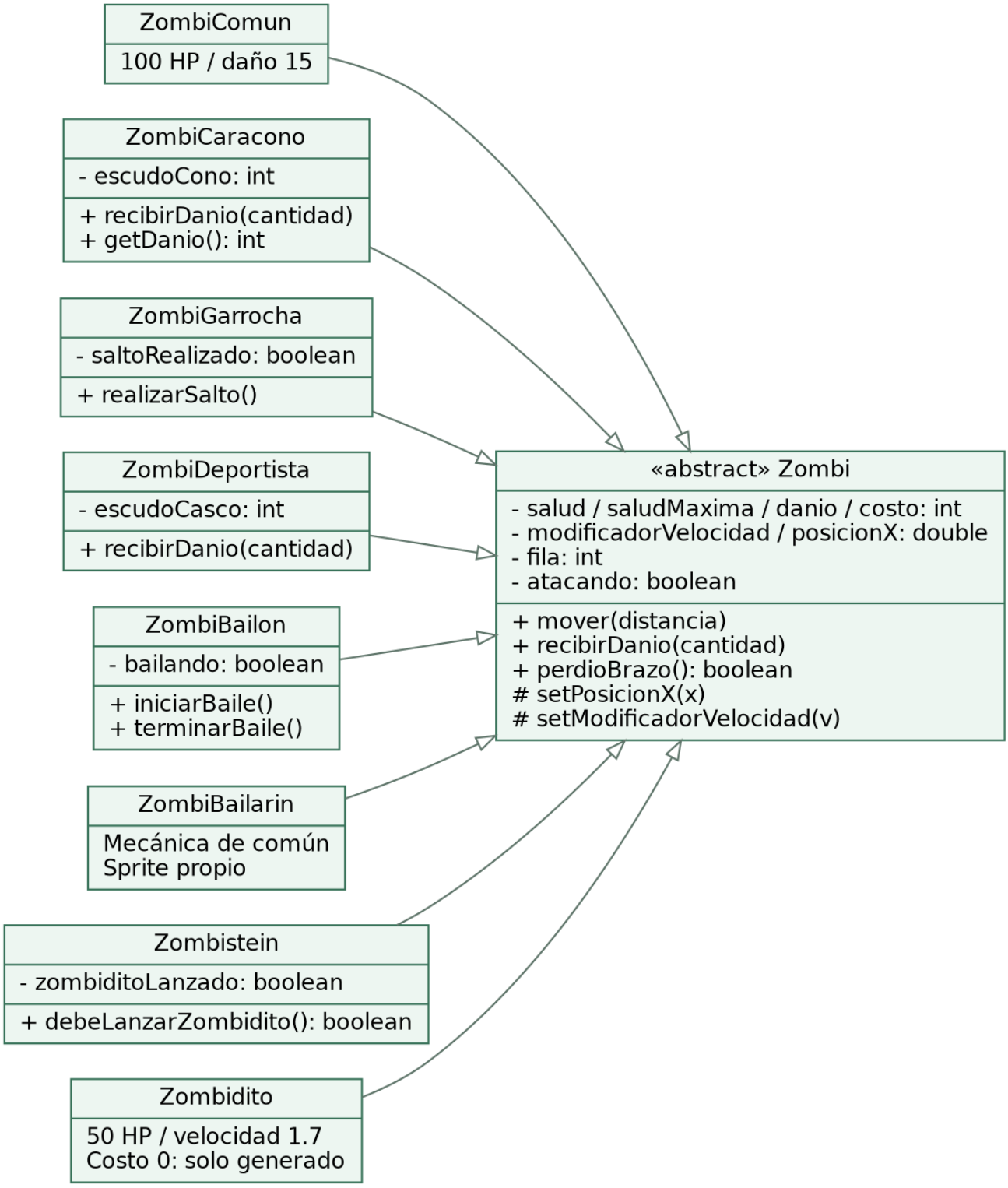


Figura T2b. Herencia de los ocho tipos concretos de zombis.

4. UML de interfaz, hilos y archivos

SeleccionCarril es una interfaz de callback: PanelJardinFondo notifica la fila y VentanaPartida recibe una lambda que solicita la compra. Serializable es una interfaz marcadora del JDK implementada por el estado persistente; no se usa para serializar ventanas o hilos.

HilosPartida administra trabajadores independientes, pero entrega sus acciones al EDT. GestorPartidas separa el formato binario de la interfaz; LectorNivel contiene un analizador privado para el formato del enunciado.

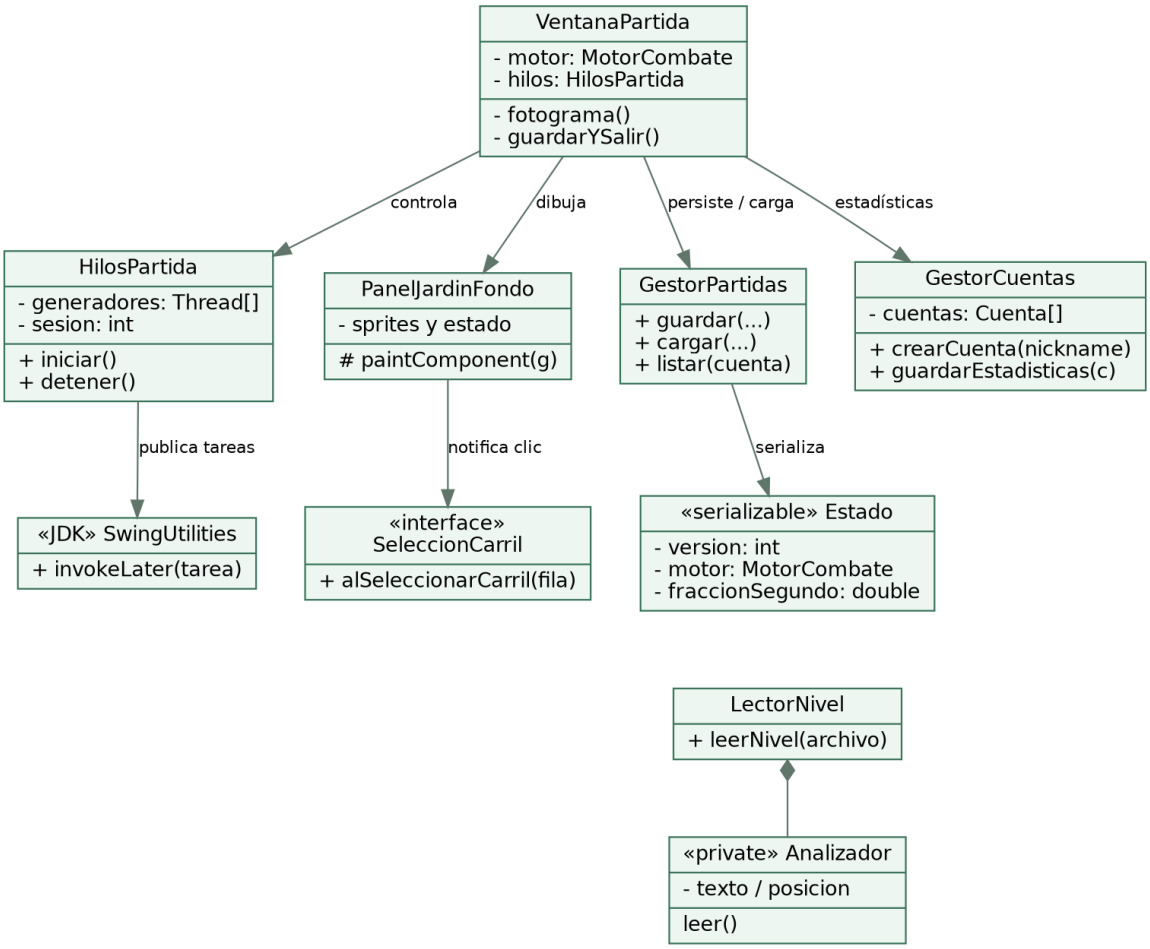


Figura T3. Relaciones de control y persistencia.

5. Flujo general del motor

La ventana mide el tiempo mediante System.nanoTime. Limita cada paso a 0.15 s y acumula fracciones para descontar segundos completos del reloj. Se aplica el mismo paso al motor para conservar la relación entre simulación y contador.

Si el reloj llega a cero, finaliza en derrota. En otro caso llama MotorCombate.actualizar, refresca paneles y comprueba si el motor detectó victoria.

```
fotograma en EDT:
  si ventana cerrada o partida terminada: retornar
  medir dt y actualizar instante
  si pausada: retornar
  limitar dt; acumular fracción y descontar segundos
  si tiempo restante == 0: finalizar derrota
  motor.actualizar(dt)
  actualizar controles y repintar
  si motor.hayVictoria(): finalizar victoria

actualizar(dt):
  si pausada/terminada o dt <= 0: retornar
  avanzar tiempo de simulación
  expirar cerebros; evaluar defensa cuando corresponda
  actualizar animación de podadoras y capacidad de arreglos
  activar Patatapum / terminar digestiones
  disparar plantas y mover proyectiles
  mover/atacar con zombis y resolver llegadas
  resolver Pinchohierba, Patatapum y Carroñívora
```

Al finalizar

La ventana marca terminada, detiene todos los trabajadores y deshabilita el catálogo. Registra una sola vez el resultado y los contadores, escribe las estadísticas, presenta el resumen y vuelve al menú. Guardar una partida no incrementa partidas jugadas: solo lo hace su conclusión.

6. Concurrencia y ciclo de vida de hilos

Trabajadores reales: cada ventana activa tiene un reloj y un Thread por generador: 6 en Fácil, 5 en Medio y 4 en Difícil. HilosPartida crea esos trabajadores como daemon. Cada uno espera 40 ms y publica su acción con SwingUtilities.invokeLater.

El reloj publica fotograma(). El generador i publica actualizarGenerador(i). Cada hilo ejecuta su propio ciclo de espera/publicación. Los trabajadores no leen salud, dinero, pausas ni arreglos del modelo: todos esos accesos se ejecutan en el EDT.

Los campos activo y sesion de HilosPartida también se leen y escriben solo en el EDT. Cada tarea captura el número de sesión con el que fue creada. Antes de ejecutarse, comprueba que siga siendo la sesión activa: así se descartan notificaciones antiguas tras una pausa o cierre.

Transición	Acciones
Iniciar	Activar modo de generadores externos; crear reloj y N trabajadores.
Pausar	Marcar partida pausada; invalidar sesión; interrumpir todos los trabajadores.
Reanudar	Conservar modelo y fechas; reiniciar instante de fotograma; crear nueva sesión e hilos.
Guardar/cerrar/finalizar	Invalidar tareas pendientes e interrumpir trabajadores.
Cargar	Deserializar datos en pausa; no iniciar hilos hasta pulsar Reanudar.

Esperas en tiempo de juego

Cada generador mantiene una fecha proximoCerebro[i] relativa al reloj de simulación. Cuando vence, su trabajador solicita crear el icono y la fecha pasa a infinito. Al recogerlo o expirar se programa otra espera aleatoria de 5–15 s. Pausar no consume esas fechas, porque el reloj del modelo no avanza.

Para pruebas sin ventanas se conserva un modo interno que evalúa generadores desde actualizar(). El modo externo es transient: nunca se guardan objetos Thread. La aplicación activa el modo externo al iniciar HilosPartida, por lo que los iconos no se generan dos veces.

7. Coordenadas, colisiones y casilla lógica

Las coordenadas del modelo no son píxeles. posicionX se mide en columnas; el centro del zombi es posicionX + 0.35. Su casilla lógica es **floor(posicionX + 0.35)**, limitada al rango 0..columnas−1. Cada entidad tiene así una única casilla para ataques de área, saltos e invocaciones.

El panel transforma estas unidades según el rectángulo de césped. El dibujo y el clic usan la misma transformación para evitar seleccionar otro carril. La casa se muestra a la derecha, reflejando visualmente el fondo original.

La velocidad base es 0.44 columnas/s multiplicada por el modificador del zombi. Una planta bloqueante limita el avance a columna − 0.30. Pinchohierba es transitable; Zombistein aplasta en vez de morder; Garrocha consume una sola oportunidad de salto y Nuez la bloquea.

Los proyectiles avanzan 3 columnas/s. Se prueba el segmento entre su x anterior y la nueva, con margen 0.14; se elige el zombi vivo más cercano del mismo carril que intersecte el segmento. Un impacto elimina el proyectil. El hielo multiplica movimiento por 0.6 durante 4 s; otro impacto reinicia la fecha sin multiplicar otra reducción.

Lanzamiento y animación

Al llegar al 50% de salud, Zombistein genera un único Zombidito. La posición de destino está antes de la penúltima casilla. Durante 0.8 s el panel interpola desde el lanzador al destino y dibuja una altura sinusoidal. Mientras vuela no muerde ni recibe ataques de plantas; al aterrizar vuelve al combate normal. Origen y fecha del vuelo son parte del estado binario. El Zombistein usa desde el lanzamiento una variante sin el pequeño en su espalda.

Garrocha mantiene por separado haber saltado y perder el brazo: se cargan variantes con/sin arma y con/sin brazo. El estado de salud no cambia por seleccionar un sprite.

8. Catálogo y reglas de las plantas

Planta	Salud	Ataque y mecánica
Lanzaguisantes	100	20 por proyectil; intervalo 1 s; dispara hacia los zombis detectados.
Patatapum	50	10 s de preparación; explosión de 100 a cada zombi de su casilla.
Hielaguisantes	100	15 por proyectil cada 1.5 s; ralentiza 40% durante 4 s.
Nuez	400	Bloquea incluso Garrocha. Estados: intacta ≥66%, grietas ≥35%, muy dañada <35%.
Carroñívora	100	Engulle en su casilla o frontal; digestión 8 s. Contra Zombistein aplica 200.
Pinchohierba	Inmune a mordidas	10 cada 0.5 s a zombis de su casilla; solo Zombistein la aplasta.
Bipetidora	100	15 por proyectil: uno frontal y dos traseros cada 1 s si hay blancos.

La salud de Pinchohierba se representa internamente con un marcador positivo, pero recibirDanio no reduce ese valor. Su destrucción por Zombistein retira la planta de la casilla directamente. Al destruir una planta se libera la casilla, se incrementa el contador y se crean 3–6 iconos de cerebro.

Las fechas preparadaEn y digestionHasta se asocian a cada casilla. Una nueva siembra reinicia esas fechas; una Patatapum de la IA espera sus propios 10 s y no utiliza la edad global de la partida.

9. Catálogo y reglas de los zombis

Zombi	HP/escudo	Costo	Velocidad	Mecánica
Común	100	50	1	Daño 15/s; brazo al 50%.
Caracono	100 + 100	100	1	Escudo primero; luego daño 15 de común.
Garrocha	120	100	1.7 → 1	Salta una vez salvo Nuez; pierde arma.
Deportista	100 + 150	150	2	Daño 10 cada 0.5 s; casco absorbe primero.
Bailón	150	150	0.8	Cada 5 s baila 1 s e invoca hasta 4 bailarines adyacentes libres.
Bailarín	100	Generado	1	Mecánica de común; sprite de bailarín propio.
Zombistein	300	350	0.7	Aplasta; al 50% lanza un Zombidito una vez.
Zombidito	50	Generado	1.7	Solo aparece por lanzamiento; no está en el catálogo.

Partida separa invocarZombi (cobra y cuenta una compra) de agregarZombiGenerado (sin cobro). Así los bailarines y Zombiditos no inflan los zombis comprados ni los cerebros gastados.

Los escudos absorben primero el daño y trasladan el excedente a la salud base. Los zombis muertos permanecen como posiciones históricas del arreglo, pero el motor y el dibujo los omiten; no vuelven a actuar.

10. Defensa de la computadora

Fácil no siembra. Medio evalúa cada 10 s, selecciona un tipo al azar y recorre columnas desde la más cercana a la casa; entre filas desempata de arriba hacia abajo.

Difícil evalúa cada 5 s y puede realizar una sola siembra. Para cada prioridad se examinan los zombis vivos antes de pasar a la regla siguiente. Si la acción no tiene una casilla válida, se continúa evaluando.

Orden	Condición	Acción
1	Zombi a ≤4 casillas de casa	Nuez o Carroñívora en una casilla libre delante del zombi.
2	Zombi rebasó una planta	Bipetidora en la primera casilla libre a su izquierda.
3	Caracono o Zombistein	Patatapum o Carroñívora delante en ese carril.
4	Deportista	Hielaguisantes en ese carril.
5	Garrocha	Nuez en ese carril.
6	Más de tres zombis en un carril	Pinchohierba en ese carril.
7	Ninguna acción anterior ejecutable	Lanzaguisantes al fondo del carril con menos defensas y espacio libre.

El enunciado menciona soles, pero no define su saldo, generación ni costos de plantas. Por ello la implementación decide por prioridad y disponibilidad de casillas; no inventa una tabla de costos de soles.

11. Hordas, podadoras y resultado

Bloque = min(7, 1 + floor((duracionTotal − tiempoRestante + fraccionReloj) × 7 / duracionTotal)). La fracción evita desplazar las fronteras al siguiente segundo entero.

Bloques 1, 2, 3, 5 y 6: precio base. Bloque 4: mitad. Bloque 7: cuarta parte. Los costos fraccionarios se redondean hacia arriba. costoActual es la única regla usada por catálogo, comprobación de saldo y cobro.

Cada carril empieza con una podadora disponible. La primera llegada la consume, elimina inmediatamente a todos los zombis vivos de ese carril y activa una animación de barrido. Una llegada posterior en ese carril marca victoria y termina la partida.

Si el reloj llega a cero antes de esa invasión, la ventana registra derrota. El resumen muestra resultado, tiempo transcurrido, plantas destruidas, zombis comprados, cerebros gastados y podadoras activadas.

Estadísticas derivadas

Porcentaje de victorias = ganadas × 100 / jugadas. Promedio de plantas = plantas destruidas / jugadas. Ambos retornan 0 si no hay partidas. La primera victoria establece el mejor tiempo; las siguientes solo lo sustituyen si son menores, incluido el caso válido de una victoria de menos de un segundo.

Cada ranking copia las cuentas elegibles a un arreglo y ordena por inserción. Rapidez es ascendente, destrucción y victorias descendentes. Los empates se resuelven por nickname. Se muestran hasta cinco; en rapidez se excluyen las cuentas sin victorias.

12. Formato del nivel JSON

LectorNivel lee caracteres mediante java.io en UTF-8, con un límite de tamaño de 4 MB. Analizador mantiene un cursor, consume espacios entre tokens y exige las claves en el orden especificado. No es un analizador JSON de propósito general: implementa el formato del proyecto.

```
{
  "duracionSegundos": 180,
  "cerebrosIniciales": 150,
  "jardin": [
    ["VACIO", "NUEZ", "LANZAGUISANTES"],
    ["PATATAPUM", "VACIO", "HIELAGUISANTES"]
  ]
}
```

Validaciones

Duración positiva; cerebros no negativos; al menos una fila y una columna; filas de igual longitud; identificadores permitidos; enteros válidos; comas/corchetes/llaves correctos; ausencia de contenido sobrante. Las cadenas no necesitan escapes para los identificadores del enunciado.

Los identificadores aceptados son VACIO, LANZAGUISANTES, PATATAPUM, HIELAGUISANTES, NUEZ, CARRONIVORA, PINCHOHIERBA y BIPETIDORA. Jardin transforma cada identificador en una planta concreta o una casilla vacía. El programa no modifica el archivo de nivel.

El analizador usa arreglos temporales que crecen al doble y después se recortan a la longitud exacta. Los espacios entre filas no cambian la estructura del jardín.

13. Cuentas y estadísticas en texto

GestorCuentas valida 1–12 caracteres alfanuméricos y busca nombres con igualdad sensible a mayúsculas. Las nuevas carpetas se codifican con prefijo u- y cuatro dígitos hexadecimales por carácter: Ana y ana siguen siendo diferentes incluso en discos Windows.

RutasCuentas reconoce también carpetas antiguas nombradas directamente con el nickname. Cada cuenta contiene un único estadisticas.txt y la subcarpeta partidas.

```
cuentas/
  u-0041006e0061/           # Ana
    estadisticas.txt
    partidas/
      AAAA-MM-DD_HH-mm-ss-SSS.bin
```

```
Partidas Jugadas: 1
Partidas Ganadas: 1
Partidas Perdidas: 0
Plantas Destruidas: 8
Zombis Invocados: 3
Cerebros Gastados: 150
Mejor Tiempo Victoria: 25
Podadoras Activadas: 1
```

Se leen las ocho líneas con BufferedReader y FileReader, extrayendo el entero después de “:”. La escritura usa FileWriter. En Java 21 la codificación por defecto es UTF-8. Primero se escribe un temporal y después se reemplaza el destino; los errores se comunican a la ventana.

14. Guardado y recuperación binaria

Solo se guarda una partida activa y pausada. GestorPartidas serializa un Estado con version = 1, MotorCombate y fraccionSegundo. El motor referencia Partida, jardín, plantas, zombis, proyectiles, cerebros, temporizadores, efectos y estado del generador aleatorio.

Los arrays de fechas conservan disparos, mordiscos, congelación, baile, digestión, preparación, vuelos y próximas siembras. Las podadoras y contadores pertenecen a Partida. Las referencias compartidas se mantienen en el grafo de serialización.

No se guardan JFrame, JDialog, Thread ni HilosPartida. El indicador de generadores externos es transient. Al cargar se comprueba tipo/version/cuenta, se vincula el objeto actual de la cuenta y se impone pausa. Los nuevos hilos solo se crean cuando el usuario reanuda.

```
Estado (version = 1)
  motor -> MotorCombate
    partida -> Cuenta, Jardin, Zombi[], podadoras, contadores
    Proyectil[], Cerebro[]
    fechas de efectos / generadores / defensa / vuelos
    Random con su estado
  fraccionSegundo
```

Seguridad e integridad

ObjectInputFilter limita clases a las del proyecto, String, Random y arreglos/primitivos, y rechaza profundidad, referencias, tamaños o bytes excesivos. El guardado se escribe primero en un temporal y luego se renombra. No se elimina el archivo anterior de una partida cargada hasta confirmar el nuevo. Ante un fallo, el juego queda pausado.

Para guardados previos a la animación aérea, los arrays nuevos de vuelo se inicializan de forma diferida. La compatibilidad se limita a esta familia de versión; no se promete leer formatos de otros programas.

15. Estados gráficos y recursos

Las 34 imágenes originales se mantienen sin cambiar su contenido. Se agregan ocho PNG transparentes para común sin brazo, Deportista sin casco, Garrocha sin arma, Garrocha sin brazo con/sin arma, Zombistein sin Zombidito, Bailarín y Zombidito.

PanelJardinFondo selecciona el sprite a partir del estado del modelo. La Nuez agrega grietas según su porcentaje de salud; Patatapum indica preparación/activación; Carroñívora indica digestión. El hielo agrega una señal azul. Los cerebros se dibujan al final y se buscan en orden inverso al hacer clic para respetar superposición.

Recursos busca primero una carpeta externa Imagenes y las rutas de recursos del proyecto; después busca dentro del JAR. La falta de un recurso deja un respaldo visual, sin alterar salud o reglas. Los botones del menú y nueva partida permanecen en 170 × 40; el selector de dificultad mide 120 × 28.



Figura T4. Comparación de estados con el renderizador real. Las variantes adicionales se crearon a partir de las referencias originales mediante una herramienta de edición de imágenes; los originales se conservaron.

16. Compilación, pruebas y verificación

Compilar: compilar.bat en Windows o sh compilar.sh en Linux/macOS. javac usa --release 21, codificación UTF-8 y ningún classpath externo. jar incluye las clases y todos los recursos con la clase principal declarada.

Pruebas: sh probar.sh ejecuta la lógica, la concurrencia y los estados gráficos. En una sesión gráfica, PROBAR_INTERFAZ=1 sh probar.sh agrega el flujo de interfaz y una partida completa.

La validación actual reúne **52 comprobaciones de lógica/persistencia, 32 de concurrencia y 22 visuales/lanzamiento**. También se recorrió el flujo crear cuenta → nivel → cerebros → guardar → cargar en pausa → derrota → estadísticas, y se obtuvo una victoria real mediante dos compras por clic y el reloj de la aplicación.

Prueba	Evidencia
Pruebas	JSON, escudos, hordas, IA, economía, pausa, podadoras, cuentas y binarios.
PruebasHilos	6/5/4 trabajadores reales, emisiones independientes, pausa, carga sin duplicados y ausencia de huérfanos.
PruebasVisuales	Selección de sprites por estado; vuelo, pausa y aterrizaje de Zombidito.
PruebasInterfaz	Compra/recolección por clic, pausa, guardado y regreso al menú.
PruebasPartidaCompleta	Primera llegada consume podadora; segunda logra victoria; resumen y persistencia.
CapturarManuales	Capturas reales y comprobación de carga exacta en pausa; partida corta con derrota.

Las pruebas se ejecutaron en Linux con JDK 21 y una pantalla virtual Xvfb. Los scripts de Windows se incluyen, pero no se ejecutaron en una máquina Windows. Las comprobaciones dan evidencia de los casos recorridos; no equivalen a certificar todos los posibles niveles o una nota académica.

Consulta guiada del código

Para explicar la implementación, sigue en orden: main/Juego → VentanaNuevaPartida/LectorNivel → Partida/Jardin → HilosPartida/VentanaPartida → MotorCombate → GestorPartidas → Estadisticas/GestorCuentas. Usa las capturas y pruebas para relacionar cada regla con un comportamiento observable.